

07 — System Images + OTA Artifact Pipeline

Revision: 1 **Parent:** 00_MASTER_INDEX.md **Prerequisites:** E-01 (RK3588 hardware), C-07 (object storage seam), F-05 (production stack deployed)

Overview

This stage produces the actual Android 15 AOSP system images for Orange Pi 5 Max (RK3588) with the OTA agent pre-installed, builds OTA artifacts from image deltas, and sets up the CI/CD pipeline for automated artifact building and publishing. This is where the OTA system becomes a REAL, FLASHABLE product.

G-01 [AGENT] — Build Android 15 AOSP System Image with OTA Agent Pre-Installed

Effort: XL **Source:** docs/research/main_specs/1.0.0-mvp/client_android/, ADR-0001

What to do:

1. Set up AOSP build environment:

- Ubuntu 24.04 host (or containerized AOSP build)
- repo sync Android 15 source for RK3588 (Orange Pi 5 Max BSP)
- Apply vendor HAL patches for the RK3588

2. Integrate OTA agent into system image:

- Build ota-android-agent APK with
android:sharedUserId="android.uid.system"
- Platform-sign the APK with the AOSP platform key

- Place APK under `vendor/helix/ota/` or `/system/priv-app/HelixOTA/`
- Include `ota-update-engine-bridge` as a library dependency
- Add SELinux policies allowing the agent to:
 - Access the network (INTERNET permission)
 - Call `android.os.UpdateEngine` (system_api permission)
 - Read `ro.boot.*` properties
 - Write to download cache

3. Configure U-Boot A/B slots:

- `fw_env.config` pointing to the U-Boot environment partition
- `BOOT_ORDER` set to try active slot first, fall back to previous
- `bootcount/bootlimit` for auto-rollback
- `helix_slot` kernel command-line parameter

4. Configure A/B partition layout:

- `boot_a / boot_b`
- `system_a / system_b`
- `vendor_a / vendor_b`
- `userdata` (shared, not duplicated)

5. Build the image:

```
source build/envsetup.sh
lunch rk3588_t-userdebug
make -j$(nproc)
```

Output: `out/target/product/rk3588_t/system.img`, `boot.img`, `vendor.img`, etc.

6. Test boot on hardware: Flash to Orange Pi 5 Max, verify:

- System boots, `adb shell` works
 - OTA agent APK is present: `adb shell pm list packages | grep helix.ota`
 - A/B slots are correct: `adb shell getprop ro.boot.slot_suffix`
 - U-Boot env is writable: `fw_printenv BOOT_ORDER`
-

G-02 [AGENT] — Platform-Sign OTA Agent APK

Effort: M **Source:** submodules/ota-android-agent/android/src/main/AndroidManifest.xml

What to do:

1. Obtain platform signing key:

- Location in AOSP tree: build/target/product/security/platform.pk8 + platform.x509.pem
- These keys MUST be secured — they sign system apps with full privileges.

2. Sign the APK:

```
java -jar out/host/linux-x86/framework/signapk.jar \  
  build/target/product/security/platform.x509.pem \  
  build/target/product/security/platform.pk8 \  
  ota-android-agent.apk \  
  ota-android-agent-signed.apk
```

3. Verify signature:

```
jarsigner -verify -verbose -certs ota-android-agent-signed.apk  
# Should show "CN=Android" (platform key)
```

4. Place in system image:

```
cp ota-android-agent-signed.apk $OUT/system/priv-app/HelixOTA/  
  HelixOTA.apk
```

G-03 [AGENT] — Generate Ed25519 Signing Keypair + Distribute Public Key

Effort: S (~15 min) **Source:** scripts/testing/gen_key.go

What to do:

1. Generate keypair:

```
go run scripts/testing/gen_key.go
# Output: private.key (64 bytes hex or base64), public.key (32
          bytes base64)
```

2. Secure the private key:

- Store in a hardware security module (HSM) or at minimum an encrypted, access-controlled file.
- NEVER commit to git (already in .gitignore — .env patterns, *.key).
- Distribute to CI/CD pipeline via secret manager (GitHub Secrets, GitLab CI Variables, or local vault).

3. Distribute public key to server:

- Set HELIX_ARTIFACT_PUBKEY=<base64-public-key> in the deploy env.
- The server reads this at startup and uses it to verify artifact signatures.

4. Distribute public key to devices (for client-side verification):

- Embed the public key in the OTA agent's res/raw/ or assets/ directory.
- Or: device fetches the public key from the server on registration (TUF-style — ADR-0002, 1.0.1+).

5. Key rotation plan:

- HELIX_ARTIFACT_PREVIOUS_PUBKEY env var for the previous key (rotation grace period).
- HELIX_ARTIFACT_SIGNING_KEY_ROTATION_INTERVAL for automatic rotation schedule.

G-04 [AGENT] — Build First OTA Artifact (Full Payload)

Effort: M (~1h) **Source:** scripts/testing/sign_artifact.go, ADR-0005 (full payload MVP)

What to do:

1. Create full OTA payload:

- For the initial release, the “artifact” is the full system image delta (or full image if it’s the first release).
- AOSP `ota_from_target_files` tool produces the `payload.bin` that `update_engine` consumes.
- Package as a `ZIP_STORED` (no compression) container with:
 - `payload.bin` — the AOSP update payload
 - `payload_properties.txt` — `FILE_HASH`, `FILE_SIZE`, `METADATA_HASH`, `METADATA_SIZE`
 - `META-INF/com/android/metadata` — OTA metadata

2. Sign the artifact:

```
go run scripts/testing/sign_artifact.go \  
  --key private.key \  
  --input system-ota-1.0.0.zip \  
  --output system-ota-1.0.0-signed.zip
```

3. Verify the signature:

```
go run scripts/testing/sign_artifact.go \  
  --key public.key \  
  --verify system-ota-1.0.0-signed.zip  
# → "Signature VALID"
```

4. Upload to server:

```
curl -X POST https://hxota.dev/api/v1/artifacts/upload \  
  -H "Authorization: Bearer $TOKEN" \  
  -F "file=@system-ota-1.0.0-signed.zip" \  
  -F 'metadata={"os_type":"android","target_model":"rk3588_t","version":"1.0.0"}'  
# → 201 {"artifact_id":"...", "verified":true}
```

G-05 [AGENT] — Create Release, Deployment, Rollout

Effort: S (~15 min)

What to do:

1. Create release:

```
curl -X POST https://hxota.dev/api/v1/releases \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"artifact_id":"<artifact_id>","version":"1.0.0","os_type":"android","tar
# → 201 {"release_id":"..."}
```

2. Create deployment:

```
curl -X POST https://hxota.dev/api/v1/deployments \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"release_id":"<release_id>","strategy":"phased","group_name":"all"}'
# → 201 {"deployment_id":"..."}
```

3. Create rollout (if phased):

```
curl -X POST https://hxota.dev/api/v1/deployments/
  <deployment_id>/rollout \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"phases":
    [{"percentage":10,"success_threshold":5,"duration_minutes":60},
    {"percentage":50,"success_threshold":50,"duration_minutes":120},
    {"percentage":100,"success_threshold":200,"duration_minutes":0}]}'
# → 201
```

4. Verify: Device polls GET /api/v1/client/update → receives update offer.

G-06 [AGENT] — CI/CD Artifact Build Pipeline

Effort: L **Source:** ADR-0001 (AOSP build), §11.4.173 (containerized builds)

What to set up:

1. Containerized AOSP build:

- Build container with all AOSP dependencies (Ubuntu 24.04, JDK, Python, repo tool)
- Mount AOSP source tree as volume or use CI cache
- Run `make -j$(nproc)` inside container
- Extract output images and OTA payloads

2. Automated signing:

- CI pipeline fetches signing key from secret manager
- Signs the artifact
- NEVER logs or exposes the private key (§11.4.10)

3. Automated upload:

- CI pipeline authenticates to ota-server using `helix-ota` CLI (from C-05)
- Uploads artifact, creates release, optionally creates deployment

4. Pipeline triggers:

- On push to `main` (for release builds)
- On tag `helix_ota-v*` (for tagged releases)
- Manual trigger (for hotfixes)

5. Artifact retention:

- Keep last N artifacts in S3/MinIO
- Tag artifacts with build number, git commit, timestamp

6. Example pipeline (GitLab CI):

```
build-ota:
  image: registry.example.com/aosp-build:latest
  script:
    - source build/envsetup.sh && lunch rk3588_t-userdebug
    - make -j$(nproc)
    - ./scripts/package_ota.sh out/target/product/rk3588_t
  artifacts:
    paths:
      - ota-package.zip

sign-ota:
  needs: [build-ota]
  script:
    - go run scripts/testing/sign_artifact.go --key
      $SIGNING_KEY --input ota-package.zip --output signed-ota.zip
  artifacts:
    paths:
      - signed-ota.zip

publish-ota:
  needs: [sign-ota]
```

script:

```
- helix-ota login --server $OTA_SERVER --username  
$OTA_USER --password $OTA_PASS  
- helix-ota upload signed-ota.zip --version $CI_COMMIT_TAG
```

G-07 [AGENT/HARDWARE] — Flash System Image + Verify OTA Agent

Effort: M (~1h) **Source:** Hardware validation

What to do:

1. Flash the system image:

```
# Using RKDevTool or fastboot  
sudo rkdevtool db rk3588_spl_loader_v1.08.111.bin  
sudo rkdevtool wl 0 system.img  
sudo rkdevtool wl 0x4000 vendor.img  
sudo rkdevtool rd
```

2. Boot and verify:

```
# Wait for boot  
adb wait-for-device  
  
# Verify Android version  
adb shell getprop ro.build.version.release # → 15  
  
# Verify OTA agent is installed  
adb shell pm list packages | grep helix.ota # →  
digital.vasic.helix.ota.agent  
  
# Verify A/B slots  
adb shell getprop ro.boot.slot_suffix # → _a or _b  
  
# Verify U-Boot env  
adb shell fw_printenv BOOT_ORDER
```

3. Verify OTA agent startup:

```
# Check WorkManager scheduled the poll worker  
adb shell dumpsys jobscheduler | grep -A5 helix.ota
```



```
# Check agent logs
adb logcat -s HelixOTA:V
```

G-08 [AGENT/HARDWARE] — End-to-End OTA Update on Real Hardware

Effort: L (~3h) **Source:** Full OTA lifecycle validation

Steps (record EACH with evidence):

1. Initial state:

- Device is on version 1.0.0 (base image)
- Device registered with server
- Device is polling for updates

2. Build OTA artifact for version 1.0.1:

- Make a small change to the system image (e.g., increment build number)
- Build the delta or full OTA payload
- Sign and upload to server
- Create release 1.0.1

3. Device discovers update:

- Device polls, receives UpdateAvailable with version 1.0.1
- Logcat shows: “Update available: 1.0.1”

4. Device downloads:

- Downloads the artifact from S3/MinIO (signed URL or direct)
- Logcat shows: “Downloading: X/Y bytes”

5. Device verifies:

- SHA-256 hash matches
- Ed25519 signature verifies
- Logcat shows: “Verification: PASS”

6. Device applies:

- update_engine writes payload to inactive slot
- Logcat shows: “UpdateEngine: IDLE → DOWNLOADING → VERIFYING → FINALIZING → FINISHED”

7. Device reboots:

- update_engine triggers reboot (or agent calls `PowerManager.reboot()`)
- Logcat shows: “Rebooting to new slot”

8. Post-boot verification:

- Device boots from the NEW slot (`ro.boot.slot_suffix` changed)
- Verified boot state is GREEN
- App version is 1.0.1

- Agent reports success telemetry to server

9. Evidence capture:

- ADB logcat dump (full OTA cycle)
 - Server logs showing device lifecycle events
 - §11.4.153 video recording (window-scoped MP4, vision-verified)
-

G-09 [AGENT/HARDWARE] — A/B Rollback Test

Effort: M (~1h) **Source:** PRODUCTION_READINESS_PLAN.md §5 P2, OTA-017

Steps:

1. Deploy a deliberately broken OTA (version 1.0.2-broken):

- The broken payload should cause the system to fail to boot (e.g., corrupt `system.img`)

2. Device applies the broken update: Same as G-08, up to reboot.

3. Device fails to boot:

- U-Boot detects boot failure (`bootcount > bootlimit`)
- U-Boot switches back to the previous slot
- Device boots from the original slot (1.0.1)

4. Post-rollback verification:

- Device is back on version 1.0.1
- Agent reports rollback telemetry event
- Server marks the deployment status accordingly

5. Evidence: Video recording of the boot failure + auto-rollback.

G-10 [AGENT] — Artifact Publishing Pipeline

Effort: M (~1h) **Source:** `scripts/remote_deploy/publish_artifacts.sh`

What to do:

1. Configure SFTP target:

- The protected download area on the remote host (e.g., /srv/artifacts/)
- Served by nginx with authentication (the server generates signed URLs)

2. Publish artifacts:

```
bash scripts/remote_deploy/publish_artifacts.sh \  
--artifact system-ota-1.0.0-signed.zip \  
--hash system-ota-1.0.0-signed.zip.sha256 \  
--dest /srv/artifacts/android/rk3588_t/
```

3. Verify:

- Artifact is reachable at the download URL
- Hash file validates the artifact integrity

4. CDN (optional, future):

- If fleet grows beyond single-host, front with CDN
- Signed URLs still work through CDN (time-limited, IP-restricted if needed)

Verification Checklist

Step	Action	Expected Result
G-01	AOSP build succeeds	system.img, boot.img, vendor.img produced
G-02	APK platform-signed	jarsigner verifies platform certificate
G-03	Keypair generated + distributed	Server has public key, CI has private key
G-04	Artifact built + signed + verified	go run sign_artifact.go -verify → VALID
G-05	Release + deployment created	GET /client/update returns offer

Step	Action	Expected Result
G-06	CI/CD pipeline works	Push triggers build → sign → upload → release
G-07	System image flashed + boots	adb shell works, helix.ota package present
G-08	Full OTA cycle on hardware	Device updates from 1.0.0 → 1.0.1 successfully
G-09	A/B rollback works	Broken OTA rolls back to previous slot
G-10	Artifacts published	Download URL serves the artifact

Danger Zones

#	Danger	Mitigation
DZ-G1	Private signing key leaked → attacker signs malicious OTA	Store in HSM or encrypted vault. Never on CI logs.
DZ-G2	OTA bricks device → unrecoverable	Always test in emulator first. Have recovery image ready.
DZ-G3	A/B slot corruption → device stuck in boot loop	U-Boot bootcount + bootlimit is the safety net. Validate it works (G-09).
DZ-G4	CI pipeline uploads unsigned artifact	Mandatory signature verification in upload handler (S3+S4+S5 reject).
DZ-G5	First OTA on a device without fallback → no rollback path	First release MUST be a full payload. Delta updates only after full payload baseline.

Honest Boundary (§11.4.6)

- The AOSP build process for RK3588 is vendor-specific and outside the scope of this OTA system. Steps G-01/G-02 assume the AOSP build environment and vendor HAL are functional.

- The CI/CD pipeline design above is a template — actual implementation depends on the CI platform (GitHub Actions, GitLab CI, Jenkins).
- G-08 (full OTA cycle) and G-09 (rollback) are the definitive proof that the system works. All prior emulator testing (Tier-0/1/2) is necessary but NOT sufficient.